

ENTREGA PROJECTE FINAL DEL CURS



Selenium
WebDriver

Índex

1.- Aplicació triada	2
2.- Decisions de disseny	2
2.1.- Tests implementats	3
2.2.- Estructura del POM.xml	4
2.3.- Enllaç al repositori	4
3.- Dificultats trobades	4

1.- Aplicació triada

La majoria dels tests s'han fet amb *The Internet* de *HerokuApp*¹.

Per fer el test del formulari s'ha intentat fer-ho amb un formulari de Google Drive², però ha estat impossible (s'entrega el codi).

El test de navegació s'ha optat per seguir l'exemple del curs i provar diferents cercadors.

2.- Decisions de disseny

En tots els casos s'ha intentat seguir una estructura POM (*Page Object Model*). Trobarem les pàgines objecte dintre del directori *pages*. A mode d'exemple:

```
public class LoginPage {
    private WebDriver driver;
    private WebDriverWait wait;

    // 1. Selectors (Locators) - Privats per mantenir l'encapsulament
    private By usernameField = By.id("username");
    private By passwordField = By.id("password");
    private By loginButton = By.cssSelector("button[type='submit']");
    private By flashMessage = By.id("flash");

    // Constructor
    public LoginPage(WebDriver driver) {
        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(5));
    }

    // 2. Accions que es poden fer a la pàgina
    public void login(String user, String pass) {

        wait.until(ExpectedConditions.visibilityOfElementLocated(usernameField)).sendKeys(u
        ser);

        driver.findElement(passwordField).sendKeys(pass);
        driver.findElement(loginButton).click();
    }

    // Mètode per agafar el text de l'alerta (tant d'èxit com d'error)
    public String getFlashMessageText() {
        return
        wait.until(ExpectedConditions.visibilityOfElementLocated(flashMessage)).getText();
    }
}
```

¹ <https://the-internet.herokuapp.com/>

² <https://forms.gle/2SkEwKsLuLYPq1NA7>

A les classes de test s'ha intentat parametritzar al màxim amb paràmetres externs, i que resten explicats al *readme*,

Paràmetre	Descripció	Exemple / Valor per defecte
<code>-Dbrowser</code>	Selecció del navegador per a l'execució del test. (Opcions: <code>chrome</code> , <code>firefox</code> o <code>chromeheadless</code>)	<code>chrome</code>
<code>-Duser</code>	Credencial d'accés per a las proves de Login.	<code>tomsmith</code>
<code>-Dpassword</code>	Contrasenya vinculada a l'usuari de les proves de Login.	<code>SuperSecretPassword!</code>
<code>-Dvalor</code>	Valor numèric objectiu on s'ha de desplaçar el component Slider.	<code>4.5</code>

Així podrem cridar al test del *slider* amb la comanda,

```
mvn clean test -Dtest=SliderTest -Dvalor=1.5
```

Tots els tests capturen evidències en cas de fallada (captura de pantalla). Es poden trobar al directori */screenshots* dintre del directori arrel del projecte.

La captura s'ha fet amb un Listeners personalitzat (TestListener.java) que es pot trobar a la carpeta *listeners* del codi.

2.1.- Tests implementats

- *BaseTest.java*: Classe base de la que hereten la resta de tests
- *LoginTest.java*: Test d'accés correcte mitjançant formulari
- *LoginTestErroni.java*: Test d'accés correcte mitjançant formulari
- *NavigationTest.java*: Test de navegació per diversos cercadors
- *RedirectTest.java*: Test de redirecció de pàgines (Opció personal)
- *SliderTest.java*: Test per comprovar el funcionament correcte d'un slider horitzontal (Opció personal)
- *FormulariTest.java***ERROR**: Intent d'utilització d'un formulari de Google

El test sobre el formulari de Google no ha funcionat, però donat que ja s'han implementat les proves sobre formularis als tests de logins, queda coberta aquesta part de l'enunciat (vegeu apartat 3).

2.2.- Estructura del POM.xml

Les proves s'han fet sobre OpenJDK 25.0.3, com es descriu al POM:

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>3.11.0</version>
  <configuration>
    <source>25</source>
    <target>25</target>
  </configuration>
</plugin>
```

Les dependències són les mateixes que hem vist durant el curs però amb l'afegit del *commons-io* per poder escriure les captures de pantalla en cas de fallada del test,

```
<dependency>
  <groupId>commons-io</groupId>
  <artifactId>commons-io</artifactId>
  <version>2.14.0</version>
</dependency>
```

Es pot veure el POM.xml complet al repositori.

2.3.- Enllaç al repositori

<https://github.com/josep-escoda/workshop-final>

3.- Dificultats trobades

La dificultat més important i que no he pogut solucionar és la interacció amb els formularis de Google, ja que s'ofusca molt el codi i gran part dels controls generen nous HTMLs flotants que són molt difícils de tractar, fins i tot, mitjançant JScript.

S'ha intentat solucionar, sense èxit, mitjançant diferents assistents d'IA, però cap ha estat capaç d'analitzar correctament el formulari per poder injectar els valors del test. També s'ha provat, de forma infructuosa, injectar aquests valors via URL.